

협업 전략 및 개발 방향

AI 기반 발표 준비 도우미 개발 회의록

July
17

회의 개요

- **일시:** 2026년 6월 28일
- **프로젝트명:** AI 기반 발표 준비 도우미
- **참여 인원:** 최국진, 이승현, 김경민
- **주요 안건:** 역할 분담, 기술 스택 확정, 시스템 아키텍처 설계, 개발 단계 수립, 리포지토리 및 Git 협업 규칙 확정

1. 프로젝트 개발 방향

본 프로젝트는 사용자가 PPT, PDF, DOCX 형식의 발표 자료를 업로드하면 AI가 자료를 분석하여 슬라이드별 발표 시간을 배분하고 발표 대본을 생성하는 서비스이다.

사용자가 스마트폰을 이용해 실제 발표처럼 연습하면 음성, 자세, 시선, 반복 행동을 분석하고, 교수·회사 상사·인사 담당자·기술 전문가 등 역할별 AI 에이전트가 발표를 평가한다.

발표 이후에는 예상 질문을 생성하고 사용자의 답변을 분석하여 실제 발표와 질의응답에 대비할 수 있도록 구현한다.

전체 개발 흐름은 다음과 같다.

발표 자료 업로드
→ 발표 자료 및 슬라이드 분석
→ 슬라이드별 발표 시간 배분
→ 발표 대본 생성
→ 스마트폰 발표 리허설
→ 음성 및 영상 분석
→ 역할별 AI 에이전트 평가
→ 예상 질문 및 답변 분석
→ 종합 리포트 제공
→ 대본 수정 및 재연습

2. 기술 스택 및 아키텍처 설계

2.1 프론트엔드

- **React Native**
- 스마트폰 카메라 및 마이크 연동
- 발표 자료 및 대본 조회
- 발표 녹음 및 영상 촬영
- 분석 결과 및 종합 리포트 시각화
- 예상 질문 확인 및 답변 녹음
- 사용자별 발표 기록과 개선 정도 비교

프론트엔드는 이승현과 김경민이 공동으로 개발하며, 하나의 React Native 프로젝트를 기반으로 기능을 분담한다.

2.2 백엔드

- **Python**
- **FastAPI**
- **RQ**
- **Redis**

FastAPI는 프론트엔드에서 요청한 발표 자료, 사용자 정보, 발표 분석 결과를 처리하는 REST API 서버로 사용한다.

RQ와 Redis는 시간이 오래 걸리는 AI 분석 작업을 비동기적으로 처리하는 데 사용한다.

주요 비동기 처리 대상

- 발표 자료 분석
- 음성 파일 변환 및 분석
- 영상 프레임 분석
- 자세 및 시선 분석
- 역할별 AI 에이전트 평가
- 예상 질문 생성
- 종합 리포트 생성

기본 처리 구조

React Native
→ FastAPI API 요청
→ Redis 작업 큐 등록
→ RQ Worker 분석 실행
→ 분석 결과 DB 저장
→ React Native에서 결과 조회

2.3 AI 및 영상 분석

- OpenAI Whisper API
- OpenAI GPT API
- YOLO Pose
- OpenCV
- MediaPipe
- FFmpeg

기술별 역할

기술	사용 목적
OpenAI Whisper API	발표 음성을 텍스트로 변환
OpenAI GPT API	대본 생성, 역할별 평가, 예상 질문 생성, 답변 분석
YOLO Pose	발표자의 신체 관절과 자세 추적
OpenCV	영상 프레임 처리 및 행동 분석
MediaPipe	얼굴, 시선, 머리 방향, 손동작 분석
FFmpeg	영상·음성 분리, 압축, 형식 변환

영상 분석 방향

영상 분석에서는 단순히 특정 행동의 발생 여부만 확인하지 않고 다음 요소를 함께 분석한다.

- 행동 반복 횟수
- 행동 지속 시간
- 행동 발생 구간
- 발표 내용과 행동의 연관성

- 이전 발표와 비교한 개선 정도

2.4 문서 분석

- **python-pptx**
- **python-docx**
- **PyMuPDF**
- **LibreOffice Headless**

문서별 처리 방식

파일 형식	처리 기술
PPTX	python-pptx
DOCX	python-docx
PDF	PyMuPDF
PPT, DOC 등 구형 문서	LibreOffice Headless를 이용한 형식 변환

문서에서 다음 정보를 추출한다.

- 슬라이드 제목
- 본문 텍스트
- 표와 그래프
- 이미지
- 발표 자료의 전체 구조
- 슬라이드 간 연결성
- 중복 내용
- 핵심 키워드
- 설명이 부족한 부분

2.5 데이터베이스 및 저장소

- **Supabase PostgreSQL**
- **Supabase Storage**
- **Supabase Authentication** 사용 여부는 개발 과정에서 검토

Supabase PostgreSQL에는 사용자 정보, 발표 프로젝트, 슬라이드 분석 결과, 대본, 리허설 기록, AI 평가 결과 등을 저장한다.

Supabase Storage에는 다음 파일을 저장한다.

- 업로드한 발표 자료
- 발표 음성 파일
- 발표 영상 파일
- 변환된 이미지 및 썸네일
- 분석 결과 파일

3. 시스템 아키텍처 설계 방향

3.1 전체 아키텍처

```
React Native App
  ↓
FastAPI Backend
  ↓
Redis + RQ Worker
  ↓
AI 및 분석 모듈
├─ Whisper API
├─ GPT API
├─ YOLO Pose
├─ OpenCV
├─ MediaPipe
└─ FFmpeg
  ↓
Supabase PostgreSQL
Supabase Storage
```

3.2 데이터 처리 원칙

- API Key를 React Native 앱에 하드코딩하지 않는다.
- OpenAI API 호출은 반드시 FastAPI 서버를 통해 처리한다.
- 사용자의 발표 자료, 영상, 음성 파일은 접근 권한을 확인한 뒤 제공한다.
- 사용자별 데이터가 분리되도록 모든 발표 프로젝트에 사용자 식별자를 연결한다.

- 영상과 음성 분석은 비동기 작업으로 처리한다.
- 분석 중인 작업은 상태값을 통해 관리한다.

작업 상태 예시

PENDING
PROCESSING
COMPLETED
FAILED

3.3 개인정보 및 파일 보안

발표 자료와 영상에는 개인정보나 회사 내부 정보가 포함될 수 있으므로 다음 원칙을 적용한다.

- 사용자 동의 없이 영상과 영상을 수집하지 않는다.
- 발표 자료, 영상, 음성의 수집 목적을 명확하게 고지한다.
- 사용자가 파일 삭제를 요청할 수 있도록 기능을 제공한다.
- 사용자별 파일 접근 권한을 분리한다.
- 파일 다운로드 URL은 영구 공개 URL이 아닌 제한된 URL 사용을 검토한다.
- 분석 완료 후 원본 영상과 음성을 자동 삭제하는 옵션을 검토한다.
- 데이터 보유 기간을 개인정보처리방침에 명시한다.

4. 역할 분장 및 리포지토리 구성

팀원은 총 3명이며, 최국진이 백엔드를 담당하고 이승현과 김경민이 프론트엔드를 담당한다.

구분	담당자	사용 기술	담당 업무
Backend	최국진	Python, FastAPI, RQ, Redis, PostgreSQL	서버 API 개발, DB 설계, 문서 분석, AI 연동, 음성·영상 분석 서버 구축
Frontend 1	이승현	React Native	발표 자료 업로드, 프로젝트 생성, 발표 대본 및 분석 결과 화면 구현

구분	담당자	사용 기술	담당 업무
Frontend 2	김경민	React Native	발표 녹음·촬영, Q&A 기능, 기록 비교 화면, 프론트엔드 PR 및 병합 관리
Common	최국진	문서 및 설계 도구	ERD, API 명세서, 화면 설계서, 개인정보처리방침, 개발 문서 관리

5. 리포지토리 구성

프로젝트는 역할별로 다음과 같이 리포지토리를 분리한다.

리포지토리	담당자	주요 내용
presentation-coach-frontend	이승현, 김경민	React Native 모바일 애플리케이션
presentation-coach-backend	최국진	FastAPI, AI 분석, 문서 분석, 비동기 작업
presentation-coach-common	공동	ERD, API 명세서, 화면 설계서, 회의록, 개인정보 관련 문서

프론트엔드 담당자 두 명은 동일한 React Native 리포지토리에서 기능별 브랜치를 생성하여 작업한다.

6. 협업 및 GitHub 관리 규칙

6.1 Pull Request 관리

- 모든 기능 개발은 별도 브랜치에서 진행한다.
- 작업이 완료되면 `dev` 브랜치를 대상으로 Pull Request를 생성한다.
- 코드 리뷰가 완료된 후 병합한다.
- `main` 브랜치에 직접 Push하지 않는다.
- 배포 가능한 상태로 검증된 코드만 `dev` 에서 `main` 으로 병합한다.

프론트엔드 PR 관리

프론트엔드 PR은 김경민이 전담한다.

김경민은 프론트엔드 코드가 병합되기 전에 다음 내용을 확인한다.

- 코드 충돌 여부
- 중복 코드 여부
- 컴포넌트 재사용 가능성
- API 요청 및 응답 처리 방식
- 예외 처리 여부
- 화면 이동 및 상태 관리 문제
- React Native 실행 여부

백엔드 PR 관리

백엔드는 최국진이 단독으로 담당하므로 자체 검토 후 `dev` 브랜치에 병합한다.

주요 기능을 병합하기 전에는 다음 내용을 확인한다.

- API 정상 동작 여부
- DB 트랜잭션 처리
- 인증 및 권한 검증
- 파일 접근 권한
- 비동기 작업 실패 처리
- API Key 노출 여부
- 오류 로그 기록 여부

6.2 브랜치 전략

다음 브랜치를 사용한다.

브랜치	용도
<code>feat</code>	새로운 기능 개발
<code>fix</code>	버그 및 오류 수정
<code>dev</code>	개발 기능 통합 및 테스트
<code>main</code>	최종 배포 버전

브랜치 작업 흐름

dev 브랜치에서 기능 브랜치 생성

- 기능 개발
 - 기능 브랜치 Push
 - dev 대상 Pull Request 생성
 - 코드 리뷰
 - dev 병합
 - 통합 테스트
 - main 병합 및 배포
-